Ambush! — Game Design Document, Version 2

Version 2 · Noah · ELVTR Game Development course, Assignment 2 · Oct 25, 2026

Version 2 of the Ambush! GDD for Assignment 2. It keeps the structure of Version 1 (submitted Oct 20, 2026), updates every section to the game as it is now built, and adds a revision log of what changed and why.

Contents

1. Executive Summary
2. What Changed and Why (new in Version 2)
3. Setting and Tone
4. Core Loop
5. Game Mechanics (5.1 Command model through 5.13 Asset roster)
6. Resources and Scoring
7. Attack Mode
8. Prototype Scope and Schedule
9. Technical Strategy: the AI dev pipeline, constraints, agent crew and token budget
10. AI Architecture: runtime agents Front and Commander

Assignment requirements map

Criterion	Where it is covered
Game Specificity	1 (summary, win and loss), 5 (mechanics), 5.11 (five named real-world levels)
Revision and Growth	2 (classmate feedback, my own design calls, agent stress-test findings and fixes)
Agent Role Clarity	9.4 (crew table: role, model, input, output), 9.5 (how agents hand off work)
Scope Realism	8 (minimum playable battle, cut list, constraints, schedule to course end)
Document Quality	Same four required headings as Version 1, plus the redline

Note on numbering: Version 1 sections 2 to 12 are now sections 3 to 10, because the revision log is inserted as section 2 and the three technical pillars are combined under Technical Strategy.

Change Log: What Changed from Version 1

Version 2 changes come from three sources: classmate feedback, my own design calls after playing the first build, and problems the agent crew surfaced while building it.

2.1 Classmate feedback

Feedback	Change in Version 2
A classmate read the game as turn-based. Version 1 said "one turn = one battle cycle", which caused the confusion.	The summary now states real-time tactics up front, with one deployment phase and no turns (1, 4).
Terrain should affect how fast units move.	Movement speed now depends on terrain and chassis: roads fast, forest and rubble slow, tracks better than wheels in mud (5.1).
How many units can a player actually manage?	A unit cap: 10 to 15 commandable ground units per side, about 50 soldiers and vehicles in the scene per side (5.2).

2.2 My own design calls

Area	Version 1	Version 2	Why
Levels	Real towns with seeded random variation each battle	Five fixed, hand-built levels from real places; no randomness at load (5.11)	Random maps were hard to test and to balance. Fixed levels let me tune each one.
Civilians	Civilians on the battlefield; your own fire could kill them	No civilians in battle. Civilian losses happen off-screen when Ukraine loses ground (6.2)	Fighting stays about soldiers; the cost of losing ground stays real.
Drones	Strikes launched from a drone button	Drones launch from a selected drone team on the map, within its range; a small off-map stock as backup (5.8)	Ties drones to units the enemy can find and kill.
Camera and HUD	Fixed zoom, a map key	Fixed zoom, edge and arrow-key scrolling, HUD fixed to the bottom 20% of the screen (5.1)	Battles are bigger than one screen; the HUD must never cover the battlefield.
Setup flow	"Prepare at HQ"	A named Setup screen: strategic map, then army selection with supplies, then the tech tree, then Start Battle (4)	Makes each decision a clear step.
Vehicles	8 per side	9 Ukrainian (Challenger 2 added), 8 Russian; Koalitsiya and BMP-3M replace Msta-S and BMP-3 (5.13)	Matches the 3D models available, and the near-future setting.

Area	Version 1	Version 2	Why
Agent models	Five agents on Haiku	Most build agents on Opus (9.4)	Haiku and Sonnet failed repeatedly on scene work and the art pipeline.
Agent coordination	Task specs and plan approval	Structured messages with proof, one shared project state file, archived old tasks (9.5)	Agents were reporting work as done when it was not.
Art ownership	One Painter for all sprites and tilesets	Painter A: vehicles, buildings and terrain. Painter B: infantry. Cartographer places buildings and terrain but never draws them (9.4)	Every kind of art has one owner, and the two painters can work in parallel.

2.3 Agent stress-test findings

Running the crew for four days, including several unattended overnight runs, exposed problems that Version 1 did not plan for. Each one changed how the crew works.

Problem	How it was found	Fix
Agents set to Sonnet were silently running on Opus. A full model name in the agent file is not accepted, so Claude Code fell back to the parent model with no warning.	I saw zero Sonnet usage on the cost tab of my dashboard.	Short model names only (opus, sonnet, haiku); added to the rules file.
QA reported success on work that was visibly wrong (camera, HUD, tank rotation).	My own playtests and screenshots.	Sentry must attach proof: Play-mode screenshots and automated tests. Messages without evidence are rejected (9.5).

Problem	How it was found	Fix
The HUD measured 18% of the screen in its test but filled 37% on screen. Five rebuilds missed it.	I kept seeing a huge HUD.	The cause was a hidden scale setting. The test now measures the HUD as drawn, at four screen sizes and while the camera zooms and pans.
16 of 17 vehicles rendered lying on their side. Four automatic orientation fixes failed.	Art review in the dashboard.	Orientation set once by a human in Blender and cached; the render script reads it instead of guessing.
The same vehicle looked bigger at some angles than others.	Art review.	One fixed crop size per vehicle across all 16 directions.
Agents redid finished work and chased stale tasks: 215 of 239 task specs were already done or obsolete.	A task kept showing as "in progress" after it was fixed.	Closed tasks archived; agents read only a short shared state file and decisions log before each task.
An agent declared a level task blocked when it was not; level scenes shipped without a camera or HUD.	Opening the levels directly.	The architect finished the task; every level now carries the full battle setup.
Two parallel Blender processes overwrote each other's data.	Missing values in the render table.	File locking on shared data; at most two Blender processes at once.

The common thread: agents are good at producing work and poor at judging it. Version 2 moves trust from what an agent says to what it can prove.

1. Executive Summary

Ambush! is a real-time tactics game about drone warfare, set in a near-future war fought across the real map of Ukraine. The player commands squads, vehicles and drone teams to hold a defensive line against waves of Russian assault, and can later go on the attack using the same mechanics with roles reversed.

It is real time, not turn-based: there is one deployment phase before each battle, then the battle plays out continuously. There are no turns, no production buildings and no resource gathering during a battle.

- **Genre:** real-time tactics, squad command (orders, not micromanagement)
- **Camera:** angled three-quarter top-down view at a fixed zoom (yaw 45°, tilt about 40°), the battlefield laid out as a diamond like classic RTS games; the player scrolls the map with the mouse at the screen edges or the arrow keys
- **Visual reference:** the clean, readable pixel art of Hyperlight Drifter, but real-world and military rather than sci-fi; vehicles are real models rendered to pixel art
- **Platform (prototype):** PC/Mac, built in Unity 6 (URP)
- **Core fantasy:** a commander making hard choices with limited men, drones and supplies, where every loss counts

How a battle is won or lost. The player wins by holding the line until the enemy assault is spent (in the current prototype: holding for 8 minutes) or by pushing the attackers back. The player loses if the enemy breaks through to the player's deployment zone and forces a retreat. Ground lost on the strategic map also costs civilian lives, counted against the score.

Status at Version 2 (Oct 23, 2026): a minimum playable battle runs in Unity: one real village level, three rifle squads, a drone team and one vehicle against two enemy waves, with the four orders, drone launches, a timer, and a win, loss and restart screen. All 17 vehicle sprite sheets are rendered in 16 directions; infantry art is next (see 8).

3. Setting and Tone

A near-future conflict fought across the real map of Ukraine and its major towns, modeled on present-day trench and drone warfare: tree lines, trenches, ruined buildings, and contested villages. The weapons are current-day equipment plus near-future concepts such as exosuits, advanced drones, and electronic warfare.

The tone is grounded and serious. The game wants the battlefield to feel like what is happening in real wars today: drones overhead, jamming, netting over trenches, small units fighting for a few hundred meters of ground. Civilians are not shown on the battlefield; the human cost of the war appears on the strategic map, where every town lost costs civilian lives.

4. Core Loop

One cycle = one battle. Each battle is fought in real time.

1. **Setup screen (before every battle):** see the strategic map, the front line and the intel report for the next battle; choose which squads and vehicles to send, paying for them and for refills from the supply pool; pick the next tech to research; press Start Battle.
2. **Deploy:** units start concealed inside the player's deployment zone.
3. **Fight (real time):** the enemy attacks the line (or the player attacks). The player issues orders; units carry them out on their own. Drones are launched from drone teams as needed.
4. **Resolve:** territory gained or lost, casualties tallied for both sides, civilian losses counted if ground was lost, veterancy updated, wiped-out units removed.
5. **Resupply:** command delivers supplies on a fixed schedule. Return to step 1.

The tension of the loop: spending heavily to win one battle leaves less for the next.

4.1 Screens between battles

The Setup screen is a separate scene that always loads before a battle. Its central view is the strategic map: the player sees where the next defensive stand will be, or selects where the next attack goes. For testing, it also has a level picker listing the five levels.

Long-range drone strikes into enemy territory are launched from the strategic map, to scout or weaken the enemy's next attack before the battle begins.

The strategic map is the real map of Ukraine with its major towns. After each battle, the runtime agent Front (see 10.1) decides how much ground changes hands and which of the five levels the next battle is fought on.

Strategic sites in Russia: the strategic map extends into Russian territory, where strategic sites can be attacked with long-range assets: long-range drones, bombs, and missiles. Hitting these sites weakens the enemy's coming attacks.

Site	Effect when hit
Ammo depot	Shorter enemy bombardments in the next battles
Airfield	Less enemy air scouting and bombing
Refinery	Fewer enemy vehicles in the next waves (less fuel)

Site	Effect when hit
Rail hub	Slower enemy reinforcement; fewer waves per battle
Command post	Enemy Commander plans less well for a time (slower, simpler tactic changes)

Effects wear off over several battles as the enemy repairs the site.

5. Game Mechanics

5.1 Command model

The player gives each unit one of four general orders: Move Forward, Hold Position, Move Back or Retreat (keys W, H, S, R, or a small command box that appears beside the selected unit). Units carry out the order on their own; how well they take cover, maneuver and fight depends on their veterancy. There is no direct control of individual soldiers. The player's decisions are what to send in first, where to hold and set up, and when to advance to take ground.

- **Forward and back** move a squad an incremental, slightly random distance, not all the way. Where it stops, soldiers take the nearest cover, beside or inside a building; with no cover, they go prone and hold.
- **Retreat** is a full withdrawal to the deployment zone; the unit is out for the rest of the battle.
- **Fighting within the order:** units never abandon their order, but they fight intelligently inside it: moving into cover ahead during a firefight, breaking line of sight when outmatched, peeking from cover to fire, and only targeting what they can damage. Vehicles fire from concealment and reverse out of sight.
- **Soldiers move as individuals:** each picks his own path, pace and cover spot, using short rushes and bounding (one or two move while the others cover). In the open they keep their spacing; they bunch up only when sharing cover. The squad stays together.
- **Terrain changes speed:** road ×1.4, open field ×1.0, forest or rubble ×0.5, trench ×0.6 (infantry only), mud ×0.9 for tracked vehicles and ×0.5 for wheeled. A squad moves at the speed of its slowest member.
- **Selection:** click a soldier or vehicle to select its unit, or drag a box to select several. Units still outside the playable area (arriving from off-map) cannot be selected.

Camera and HUD. The battle view uses one fixed zoom, high enough to see several squads at once. The player scrolls by pushing the mouse to the screen edge, with the arrow keys, or by clicking the minimap. The spacebar opens a full battlefield map with fog of war: what your units and scout drones can currently see, and what is hidden. The HUD is a single bar fixed to the bottom 20% of the screen at any window size, like a pilot's visor: camera zoom and scrolling never change it. Left: the selected unit and its orders. Center: drone buttons with counts. Right: score, supplies and minimap.

5.2 Squads

- The basic unit is a squad of five soldiers.
- Every soldier has his own health bar. When soldiers die, the squad shrinks and gets weaker.
- Squads start generic and are upgraded with abilities. An upgrade turns one soldier into a specialist, with a maximum of one specialist per squad.
- The rest of the squad supports the specialization. Example: a heavy weapons upgrade gives the squad a heavy machine gun and anti-tank capability.
- **Unit cap:** each side has 10 to 15 commandable ground units (squads and vehicles together), about 50 soldiers and vehicles in the scene per side. Enemy waves respect the same cap.

5.3 Unit roles

Every squad starts as riflemen. Riflemen are then upgraded into one of the other roles.

Role	How you get it	What it is
Rifleman	Base unit	Standard infantry; the starting point for every squad
Exosuit trooper	Upgrade from Rifleman	Heavy infantry and shock troops: better armor, health and firepower
Heavy weapons	Upgrade from Rifleman	Machine guns and anti-tank weapons; suppression and the main infantry answer to armor
Drone operator	Upgrade from Rifleman	Short-range drone strikes and recon from the back line

Role	How you get it	What it is
Engineer	Upgrade from Rifleman	Sapper work: builds fortifications and netting, demolitions, electronic warfare

Tech tree: tech advances with time only, at no supply cost; the player picks the next tech on the Setup screen. There are four branches, each with four tiers.

Tier	Drones and missiles	Infantry	Vehicles	Economy
1	Drone operator role; faster drone loading; ground drone with a machine gun or recovery kit	Heavy weapons and Engineer roles	Cage and standoff armor	Cheaper squad refills
2	Fiber-optic upgrade; +1 drone per team; anti-tank weapon for ground drones	Body armor (more soldier health)	Air defense (Gepard / Tunguska)	Faster resupply schedule
3	Loitering munitions; tighter strike area	Better cover use and maneuvering	Heavy tanks and self-propelled howitzers	Larger supply pool
4	Swarm control (+2 drones per team); long-range strikes	Exosuit trooper role	EW truck	Long-range strike budget on the strategic map

5.4 Vehicles

Vehicles are standalone units with their own crew. They take the same four orders as squads. Squads can ride inside armored transports and infantry fighting vehicles, one squad each. Ukrainian vehicles are few, reflecting the real war: by default one infantry fighting vehicle and one anti-air vehicle, sometimes a tank, starting on the map in concealed positions. Russian vehicles start off the map and drive in along roads from the far edge, staggered. See 5.13 for the roster.

5.5 Hard counters

Units have clear strengths and weaknesses. Rifle squads do no damage to tanks; armor must be handled with anti-tank weapons (a heavy weapons squad), drones or other armor. Anti-air vehicles shoot down drones. The QA agent treats any break in these rules as a bug.

5.6 Veterancy

- Squads and vehicles gain veterancy the longer they survive in combat.
- Higher veterancy means faster reactions, better cover choice and target choice, better survival, and more effective fighting.
- Reinforcing a squad keeps its veterancy. A unit that is completely wiped out is gone, along with all its veterancy.

5.7 Retreat

There is no morale system; the player decides when to retreat. A retreated unit is out for the rest of that battle and fights another day. Injured squads must either keep fighting or fall back to a rear position and fight from there. There is no reinforcing during a battle.

5.8 Drones and reserve munitions

Drone combat is the core pillar of the game. Drones are powerful but scarce, so battles are won by combining them with squads and vehicles.

- **Drones come from drone teams on the map.** Each team carries its own small stock (for example 3 FPV drones), shown above the team. Teams operate concealed from the back line and fight with rifles if engaged. They can be found and destroyed; a team that is wiped out loses its drones.
- **Launching:** click a drone team to select it. The HUD shows that team's drones and its range circle. Click a drone type (or press 1 to 4), then click a point on the map inside the range. The drone takes off from the team's position, flies to the target area and tries to hit what is there. The count goes down; a button greys out at zero. Right-click cancels.
- **Multi-launch:** shift-click several teams and launch one drone from each at the same area, to overwhelm air defense and jamming. A team that is moving or retreating cannot launch.
- **Off-map stock:** a small arsenal of drones can also be launched from the player's map edge, with no range limit, as a backup.

- **Readying and accuracy:** a new team readies and flies one drone at a time; operator skill, veterancy and tech decide how fast it readies drones and how many it can fly at once. The target is an area, not a point, and drones do not always hit.

Drone	Range	Main use	Targets
FPV kamikaze	Short	The most-used drone on the battlefield; one-way strike	Infantry, light vehicles, positions
Scout drone	Short to medium	Spots enemy units and guides strikes	Reveals targets; no damage
Loitering munition	Medium	Circles an area, then dives on a target it finds	Vehicles, artillery, tanks
Long-range strike	Long (strategic map)	Strikes inside the enemy country	Logistics and assets behind the front

Fiber-optic upgrade: FPV and scout drones can be upgraded to fiber-optic control, which makes them immune to jamming but limits their range.

Ground drones: unmanned ground vehicles controlled by drone teams. At tech tier 1 a ground drone carries either a machine gun or a recovery kit that pulls your own losses, or enemy losses, off the field. Tech tier 2 adds an anti-tank weapon.

5.9 Drone counters

All six real-world counters are included:

- **Electronic warfare jamming:** an EW specialist or EW truck creates a zone where drones lose control.
- **Shooting them down:** small arms at close range with poor odds; anti-air vehicles are much better.
- **Physical protection:** sapper-built netting over trenches and roads; cage armor on vehicles.
- **Concealment:** units under tree lines or in trenches are harder to spot.
- **Interceptor drones:** drones that hunt enemy drones.
- **Striking operators:** only possible against on-map drone teams; off-map drones have no operators to hit.

5.10 Terrain and fortifications

- Maps include buildings, tree lines, hedgerows, trenches, roads, fields and water.
- Some maps have fixed, indestructible fortifications.
- Engineers can build new fortifications, which are destructible.
- Each terrain type looks distinct at a glance: buildings grey-brown with darker roofs, tree lines dark green, trenches dark brown, open fields light tan. Ukrainian units are blue and Russian units red-brown.

5.11 Level design

The game ships with five fixed, hand-built levels, each based on a real place in eastern Ukraine and its OpenStreetMap layout. Each level is built once and saved; nothing is randomized when it loads. Three are urban and two are open terrain.

Level	Real location	Type	What defines it
1. Village edge	Ivanivske	Urban	Village outskirts: open fields, tree lines, a road, houses at the edge
2. Town centre	Soledar	Urban	Streets and blocks; close fighting between buildings
3. Industrial	Sievierodonetsk, Azot plant	Urban	Factory buildings, rail yard, long sight lines
4. River crossing	Siverskyi Donets river	Terrain	Open fields, tree lines and a river crossing
5. Wooded hills	Serebrianske forest	Terrain	Forest, a ravine and one road

Rules every level follows:

- Playable area at least 900 × 900 m, with terrain continuing about 200 m past every edge, so the world never ends in a hard line. Units arriving from off-map drive through this border and can only be commanded once inside.
- The player starts in a deployment zone at one corner and the enemy at the opposite corner, at least 600 m apart (about 1,100 m in the current levels). The ground between them is clear at the start.
- The camera starts on the player's zone; the player scrolls to see the rest.

- Each level is saved as a Unity scene plus a JSON file with its name, real location, map source, size, start zones and cover points, and each can be played on its own.

Workflow: pick the real place, simplify its OpenStreetMap layout into a JSON layout, build the scene from it (Cartographer agent), then play-test sight lines, cover and attack routes and adjust.

5.12 Enemy assault structure

An enemy attack comes in waves. The number varies with how the battle is going; some battles are a single wave. Each battle runs in five phases:

1. **Intel:** a short report warns that enemy forces are approaching. A calm before the storm while the player's units sit concealed.
2. **Enemy scouting:** enemy drones scout the line. Each of the player's units may or may not be spotted, by chance. An icon above each unit shows whether it is concealed or revealed.
3. **Bombardment:** artillery or drone strikes on revealed positions, while the player holds in cover.
4. **Vehicle advance:** enemy armor drives in from off-map. The player answers with vehicles, drones and anti-tank weapons. Easier scenarios may open here.
5. **Mechanized infantry:** transports bring infantry forward, stop just outside the player's ground weapons' range (drones can still reach them) and unload.

This order teaches the player to read the attack: stay concealed through the bombardment, save anti-tank weapons and drones for the armor, then hold the line against infantry.

How a battle ends: **victory** if the player holds the line and the attack is spent or pushed back (in the prototype, holding for 8 minutes); **defeat** if the enemy breaks through to the player's deployment zone and forces a retreat. A result screen shows the score and casualties, with a Restart button.

5.13 Asset roster

Everything the player can deploy. The roster is kept small on purpose. Vehicles are real-world models used in the war in Ukraine.

Infantry squads (5 soldiers, one specialist at most)

Squad	Specialist	Strengths	Weaknesses
Rifle squad	none	Cheap, flexible, holds trenches	Cannot hurt armor
Exosuit squad	Exosuit trooper	Tough shock troops, upgraded stats	Expensive
Heavy weapons squad	MG / anti-tank gunner	Suppression, kills vehicles	Slow, needs cover
Drone team	Drone operator	Drone strikes and scouting from the back line; ground drones	Weak if found
Engineer squad	Sapper / demolitions / EW	Builds fortifications and netting, mines, jams drones	Poor fighters

Vehicles (own crew; separate models per side with similar stats)

Class	Vehicle	Role	Ukrainian side	Russian side
Heavy armor	Heavy tank	Slower, bigger firepower, longer range	Leopard 2A6, Challenger 2	T-90
Heavy armor	Main battle tank	Faster, shorter range; leads assaults and flanks	M1A2 Abrams	T-72B3
Light armor	Self-propelled howitzer	Long-range bombardment from behind the line	PzH 2000	2S35 Koalitsiya-SV
Light armor	Infantry fighting vehicle	Carries one squad, autocannon support	M2 Bradley	BMP-3M
Light armor	Anti-air vehicle	Cannon platform that shoots down drones	Gepard	2K22 Tunguska
Light / transport	Armored transport	Carries one squad, lightly armed	MRAP	BTR-82A
Light / transport	EW truck	Wide-area drone jamming; unarmed, priority target	EW truck	EW truck

Class	Vehicle	Role	Ukrainian side	Russian side
Light / transport	Technical	Fast armed pickup for raids and quick anti-drone fire	Pickup with ZU-23-2	Pickup with DShK

Vehicle armor upgrades: all heavy and light armor can be upgraded with cage armor or standoff armor, giving extra protection against drones and anti-tank weapons (deferred past the prototype).

Arsenal (limited, called in mid-battle)

Asset	Effect
Off-map strike drones	Fly in from the player's map edge and attack a target; can miss or be countered
Interceptor drones	Hunt enemy drones
Artillery barrage	Area bombardment
Reserve munitions	Extra ammo and anti-tank rounds for the line

6. Resources and Scoring

Supplies (what you fight with) and score (how you are judged) are kept separate.

6.1 Supplies

- A finite supply pool from military command pays for units, refills, drones and munitions on the Setup screen.
- Resupply arrives on a fixed schedule.
- Squads are refilled between battles, which costs supplies.
- Overspending in one battle means weaker or more expensive options in the next.
- During a battle, supplies are shown on the right of the HUD.

6.2 Score

- Territory gained or lost.

- Casualties: fighters lost on both sides, tallied after each battle. There are no civilians on the battlefield.
- Civilian losses: when Ukraine loses a battle and ground is lost, civilian losses are calculated from the territory lost (more ground, more civilians) and added to the score. Holding or winning costs no civilian lives.

7. Attack Mode

The game is defense-focused, but the player can also attack. Attack mode uses the same systems with roles reversed: the player leads the assault and the computer defends its line. Attack mode is after the prototype (see 8).

Assault sequence:

1. **Scouting:** drones or aircraft locate enemy positions, fortifications and anti-tank threats.
2. **Bombardment:** artillery, bombing and drone strikes soften the enemy position to break their line.
3. **Main force:** the player commits a chosen set of assets, leading with vehicles or with infantry, then adapts from there.

Winning an attack: break through the enemy line and force them to retreat, gaining territory. Losing: the assault stalls or is pushed back, at a cost in men and supplies.

8. Prototype Scope and Schedule

The target for the end of the course (November 2026) is one complete, good-looking battle, not the whole game. Everything else is cut until that battle is fun to play.

8.1 Minimum playable battle (built)

The minimum playable battle passed all six of the QA agent's Play-mode checks on Oct 23, 2026:

1. The village-edge level loads, and the camera starts on the player's zone.
2. The player has 3 rifle squads, 1 drone team and 1 vehicle (placeholder art).
3. Forward, hold, back and retreat work on squads and the vehicle.
4. An FPV drone button with a count: arm it, click the map, the drone flies, hits or misses, and the count goes down.

5. Two enemy waves (infantry, then infantry plus a tank) enter from the far side.
6. Holding for 8 minutes wins; enemies reaching the player's zone lose. A result screen shows score and casualties, with Restart.

A playtest round on Oct 24 found 10 problems (drone feel, enemy advance, wave timing, unclear feedback); all 10 were fixed, and the level gained an intel briefing, a how-to-play screen and a result screen. What is still rough: the finished vehicle sprites are not yet switched on in the level, soldiers are placeholder shapes, the ground has no texture, and tactics are basic.

8.2 Milestones to course end

In order, each one playable before the next starts, each ending with my own playtest:

1. **Core battle polish:** HUD, camera, selection and orders reliable at every screen size; readable units and terrain.
2. **Real art in the battle:** the 17 vehicle sprites (16 directions each) and the infantry sprites (8 directions; idle, walk, fire, prone) placed in the village level, with a ground texture.
3. **Combined arms:** vehicles with hard counters, transports carrying squads, anti-air, drones launched from drone teams.
4. **Full battle sequence:** intel, enemy scouting with concealed and revealed icons, bombardment, vehicle advance, infantry unloading, fog of war and the battle map.
5. **Stretch:** the other four levels made playable with the same systems.

8.3 Cut from the prototype

- Attack mode
- The strategic map campaign and long-range strikes on sites in Russia (the Setup screen exists; the campaign behind it does not)
- The runtime agents Front and Commander; their rule-based fallbacks stand in for them
- Civilian loss calculation on the strategic map
- Tech tree effects beyond the basic unlocks
- Drone counters beyond jamming and anti-air
- Cage and standoff armor, veterancy, the exosuit squad

8.4 Constraints

- **One developer,** part-time, about five weeks left in the course.

- **Usage limits:** the build runs on a flat-rate Claude plan with rolling limits. Long unattended runs hit rate limits; heavy art and scene work on Opus uses limits fastest.
- **One laptop:** 24 GB of RAM, so at most two Blender renders at once, never while Unity is in Play mode, and at most four agents in parallel.
- **Art takes the longest:** every vehicle needs a licensed 3D model, a correct orientation and a review pass. Infantry models are still to be sourced.
- **Human review is the bottleneck:** art approval and playtesting can only be done by me, so autonomous runs stop and wait at those points.

9. Technical Strategy

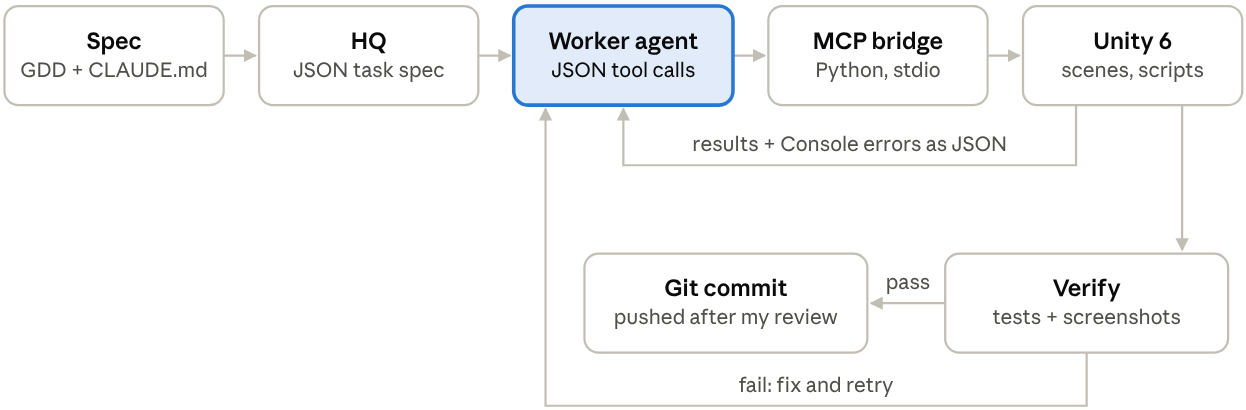
The game is built by a crew of AI agents in Claude Code that I direct. This section covers the three required pillars (the AI dev pipeline, technical constraints, token budgets) and the agent crew itself.

9.1 The AI dev pipeline

This is how a design decision becomes a working object in Unity.

1. **Spec (raw text):** this GDD plus a project rules file (CLAUDE.md) in the repo. CLAUDE.md is what the agents read; every design change goes there first and gets a line in a change log, so this GDD can be updated from it.
2. **AI node (agent):** the architect agent, HQ, reads the spec, breaks a feature into small tasks and writes each as a JSON task spec: goal, files, assigned agent, a "done means" checklist and the test to run. I approve the plan.
3. **Translation (JSON):** each step becomes a tool call sent as structured JSON over the Model Context Protocol (MCP): create a GameObject, add a component, write a C# script, set a value.
4. **Bridge:** the MCP for Unity server (Python, stdio transport) relays the JSON to the Unity Editor plugin on a local port.
5. **Engine (Unity 6):** the plugin executes the command. Unity returns results and Console errors as JSON to the agent.
6. **Verify:** automated Play-mode tests and Play-mode screenshots check the result against the task's checklist. Accepted work is committed to Git; I push after review.

A design change reaches Unity as JSON, and errors flow back to the agent



AI dev pipeline · spec to Unity and back

The worker never sees Unity directly: every change goes through the bridge as JSON, and every result or error comes back the same way, so the agent can fix its own mistakes before anything is committed.

Data-driven content: unit stats, vehicle classes, terrain speeds, drone stocks, the tech tree and level layouts live in JSON files. Agents edit these files, and Unity reloads them, including while the game is running, so values can be tuned live without touching code.

Art pipeline: licensed 3D models (Sketchfab, CC-licensed, sources logged) are rendered by Blender running headless with a Python script: each vehicle turned through 16 directions at the game's camera angle, then shrunk to pixel art at real-world scale with one fixed crop per vehicle, full color, no outline and a soft drop shadow. Camo tint is a separate final step, so vehicles can be recolored without re-rendering. Soldiers follow the same pipeline in 8 directions. Buildings are rendered the same way from simple 3D models sized to their OpenStreetMap footprints, and terrain (ground, roads, tree lines, trenches, water) is made as tiles in the same palette, so every level shares one look. Finished sprite sheets go to a review page where I approve them or send them back with a note.

Known silent-failure points: Unity not running or the bridge session inactive (calls find no editor); the agent tool list only loads when Claude Code starts; malformed JSON data; agent model names written in the wrong form (they silently fall back to the parent model); tests that pass while the screen shows something else. Each now has a check.

9.2 Technical requirements and constraints

Development environment: MacBook Pro, Apple M5 Pro, 24 GB RAM, macOS; Unity 6 (URP), new Input System; Claude Code with the MCP for Unity bridge; Blender 5.2 (headless); Python 3.12; Git and GitHub.

API and usage limits: development runs on a flat-rate Claude plan with rolling usage limits rather than per-call billing. Long autonomous runs do hit temporary rate limits; agents then pause and resume. Any scripted pipeline using the API directly has a monthly spending cap.

Context window: an agent can only hold a limited amount of project text at once. What follows from that:

- Each agent starts a task by reading only two short files: the shared project state and a decisions log of under two pages. Old reports, logs and finished tasks are not loaded.
- Closed task specs are archived (220 of 240 so far), so agents do not chase stale work.
- Unit and balance data live in small JSON files read on demand.
- Long sessions are cleared between milestones.

Processing latency: each build step takes seconds to minutes, which is fine at build time. At runtime no LLM call ever sits in the frame loop (16 ms target): Front runs between battles, and Commander only sets high-level plans in the background while behavior trees move units every frame.

Machine limits: at most two Blender renders at once, never while Unity is in Play mode; at most four agents in parallel, two of them painters.

Runtime models: both runtime agents (section 10) use a small open-source model running locally through LLMUnity, not a cloud API: no API key in the game, no per-player cost and no internet requirement. The trade-offs are 4 to 8 GB of memory, a larger download, weaker reasoning, and competition for the graphics card. Every runtime agent has a rule-based fallback.

9.3 Token budgets and projections

Version 1 projected 350 to 900 million tokens for the whole build, about $115 to $300 at API prices, assuming five of six agents on Haiku.

What actually happened in the first build week (approximate, from ccusage and my dashboard's cost tab):

- About 1 billion tokens in seven days, almost all on Opus. The dashboard estimates this at several hundred dollars at API prices. My real cost is the flat monthly plan.
- About 97% of tokens are cache reads: agents rereading context. The main architect session used far more than the worker agents.
- The workers ran on Opus instead of Haiku or Sonnet, partly by choice (Haiku and Sonnet failed repeatedly) and partly by accident (the model-name bug in 2.3).

What this changes:

- The Version 1 blended rate of about $0.33 per million tokens no longer holds with Opus workers. At API prices the full build would cost several times the Version 1 estimate.
- To bring it down: return routine tasks (data files, simple scripts, levels) to Haiku or Sonnet once the systems are stable; keep agents on the two short state files instead of long histories; archive finished work; use plain scripts for repeat jobs such as rendering and validation.
- I revise this projection at each milestone from measured usage.

9.4 Agent crew

The crew is hierarchical: HQ plans and delegates, the workers build, Sentry checks, and I approve at fixed gates. Workers never hand work to each other directly.

Agent	Role	Model	Input	Output (format, location)
HQ	Architect: plans, splits work into tasks, reviews results, keeps project state	Opus	GDD, CLAUDE.md, project state, worker messages	JSON task specs in .claude/tasks; the shared project state and decisions log
Forge	Gameplay code	Opus	Task spec plus only the files it names	C# scripts that compile with zero errors, plus a Play-mode test per feature
Ledger	Game data	Haiku	Task spec, existing JSON	JSON data files that pass a schema check

Agent	Role	Model	Input	Output (format, location)
Painter A	Vehicle, building and terrain art	Opus	Licensed 3D models, locked art settings	Vehicle sprite sheets (16 directions); building sprites and terrain tiles (ground, roads, tree lines, trenches, water); a review image for each
Painter B	Infantry art	Opus	Licensed 3D models, locked art settings	Infantry sprite sheets (8 directions, 4 poses) plus a review image
Cartographer	Levels	Haiku	OpenStreetMap reference, level spec	Level scene plus JSON manifest (size, source, start zones, cover); places Painter A's building and terrain art
Sentry	QA	Opus	Task checklist, Console, test results, screenshots	Fixed report: each checklist item pass or fail, screenshot paths, verdict
Playtest	Milestone playtester	Sonnet	The current build	A played battle and a bug report with screenshots, once per milestone

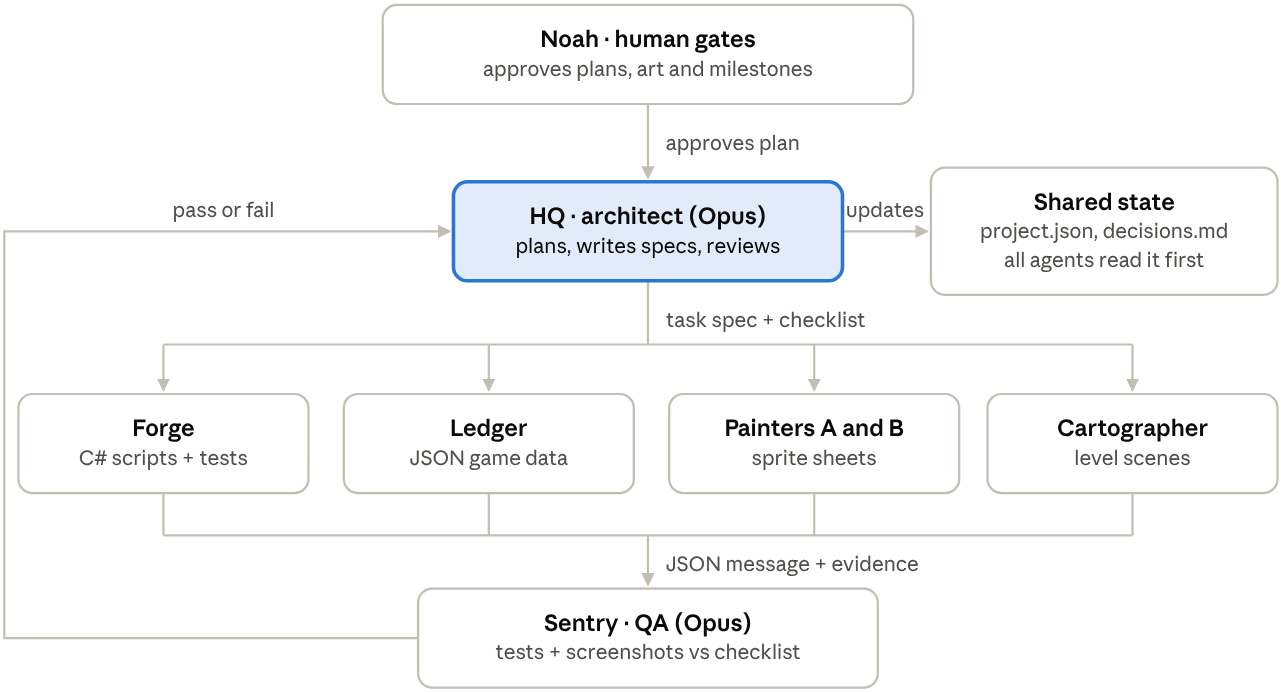
Rules that keep roles separate:

- HQ never writes game code; a task touches at most 10 files, all of a type the assigned agent is allowed to edit.
- Forge writes scripts only, never scenes, prefabs or data. Cartographer owns scenes, through the MCP tools only, never by editing scene files by hand. Cartographer places buildings and terrain but never draws them; Painter A makes all building and terrain art.

- Before changing a scene or config, a worker records its current state in its handoff message.
- Workers may not edit CLAUDE.md, the agent files or the project state; they ask HQ instead.
- Sentry reports and never edits.

9.5 How work moves through the crew

Every task goes out from HQ and comes back through Sentry with proof



Fails twice: retried once on Opus, then logged as blocked

agent crew · one architect, five workers, one checker

Workers never hand work to each other: every task goes out from HQ and comes back through Sentry with proof before it counts.

- **Task spec:** HQ writes each task as JSON with a "done means" checklist.
- **Structured handoff:** every worker returns a JSON message with a fixed format: task, agent, status (done, failed or blocked), outputs, evidence and notes. HQ rejects any message without evidence, such as a test result or a screenshot.
- **Shared state:** one project state file lists assets, approvals, blocked items, decisions and open tasks. Every agent reads it before a task; only HQ writes it.

- **Escalation:** a task that fails twice is retried once on Opus; if it still fails, it is logged as blocked and the run moves on.
- **Inbox:** I can message a single agent through an inbox file; HQ delivers it with the next task.
- **QA in three tiers, to save tokens:** automated Play-mode tests after every task (no AI tokens); one screenshot check by Sentry at the end of each feature; a full playtest by the Playtest agent at the end of each milestone.
- **Human stage gates:** I approve each milestone's plan, approve art on the review page, and playtest at the end of each milestone. A local dashboard shows which agent is working on what, the task handoffs, the event log and the token cost.

10. AI Architecture

Two runtime agents run while the game is played: Front runs the campaign between battles, and Commander sets battle tactics for both sides. Both are after the prototype; until then their rule-based fallbacks stand in.

10.1 Runtime agent Front

Front is the campaign agent: it runs the war on the strategic map between battles and adds unpredictability to the campaign, so no two playthroughs follow the same path. In Version 2 it no longer generates maps; it chooses which of the five fixed levels each battle is fought on.

Item	Detail
Input	Strategic map state (real Ukraine map, major towns, current front line), the battle result and score, territory held by each side
Output	JSON: territory gained or lost, the new front line, where the next battle is (attack or defense), and which of the five levels it is fought on
Model	Open-source model (e.g. Qwen or Llama, 3 to 8B parameters, quantized) running locally on the player's computer through LLMUnity

How it decides:

- **Territory swing:** the battle score sets the direction and base size of the swing; the agent adds a random factor so results are not fully predictable. A narrow win may gain little ground; a collapse may lose a whole town. Ground lost sets the civilian losses for the score (6.2).
- **Next battle:** the agent picks where on the front the next fight happens, and the level that best matches the terrain there: village, town, industrial area, river or forest.

Guardrails: the output is schema-validated before the game uses it; the territory swing is clamped to a maximum per battle; the next battle must be on or next to the current front line; if the local model fails, is too slow or returns invalid output, the game falls back to a rule-based result from the score and a seeded random roll, so the campaign never stalls.

What the player sees when it fires: after the results screen, the strategic map updates. The front line moves, towns change hands, and the next battle location and level are marked.

Token budget: about 5K tokens in and 1K out per call, one call per battle. Because the model runs locally there is no per-call cost; the budget limits wait time instead. One call takes a few seconds and is hidden behind the results screen.

10.2 Runtime agent Commander

The second runtime agent controls battle tactics for both sides. It is split into two layers so the game stays fast:

Layer	What it does	Runs on	How often
Commander (LLM agent)	Big decisions: plans enemy waves, changes enemy tactics when the player's line holds, turns the player's general orders into a plan for each squad	Same local open-source model as Front	At each wave start and every 20 to 30 seconds, or when something major happens (a squad wiped out, the line broken)
Unit AI (behavior trees)	Moment to moment: moving, taking cover, breaking line of sight, picking targets, firing	Unity, no LLM	Every frame

- **Enemy side:** at the start of each wave the Commander reads the map, the visible player positions and the wave budget, and outputs a plan (routes, which units lead, where to bombard). It revises the plan when the attack stalls or breaks through.
- **Player side:** when the player gives forward, hold, back or retreat, the Commander turns it into concrete goals for each squad (which tree line to occupy, which building to clear), adjusted for veterancy. The behavior trees carry those goals out.
- **Output:** JSON orders per squad or vehicle (destination, stance, priority target). The behavior trees only accept orders that are valid on the map.

Guardrails: the game never waits for the Commander. Units keep executing their last orders until a new plan arrives. If a call fails or the output is invalid, a rule-based planner takes over. The enemy Commander only sees what its own units have spotted, so it obeys the same fog of war as the player.

What the player sees when it fires: the enemy shifts its attack to a weak point after a failed push; your squads spread across the right cover after a single order.

Token budget: about 4K tokens in and 500 out per call, roughly 20 to 40 calls per battle (about 100 to 180K tokens per battle). Running locally there is no per-call cost, but each call competes with the game for the graphics card. Calls run in the background at low priority, and the Commander calls less often on slower machines; below a minimum spec, the rule-based planner runs instead.